WASP Software engineering assignment

Hannes Marklund
hannes.marklund@umu.se

August 2026

1 Introduction

In short, my research is about calibrating coarse robot or vehicle simulations using data from the real world. I specialize in simulations composed of rigid bodies, mechanical constraints (joints, contacts, and friction), and drivelines (shafts, gears, motors, and hydraulics), an area known as multibody dynamics. I focus on a formulation in which mechanical constraints are enforced using additional variables called Lagrange multipliers and regularized to model non-ideal (soft) constraints. The calibration is typically performed through the numerical optimization of state variables and parameter variables that appear in the multibody model.

I am interested both in algorithmic details, where the intersection of multibody dynamics and system identification gives rise to new perspectives and possibilities, and in the complete tuning pipeline.

Since my research is mostly based on numerical experiments performed on a computer, although I also conduct some real-world experiments and data collection, software engineering is directly applicable to my project.

I have developed two larger software projects for my research: a parameter-optimization engine and a multibody-simulation engine. The motivation for these custom implementations is to have full control over the optimization algorithms, allow function transformations such as computing derivatives.

While my research is not software for AI, there is a similarity in that optimization is used to tune parameters in a model. The difference is between black-box and white-box optimization: my goal is not merely to find a set of parameters that fits the data, but to find parameters with physical meaning. As a consequence, uncertainty quantification becomes important.

Although my software projects are still small and limited in scope compared with larger software projects involving many people, I find that many of the topics discussed in the lectures apply to my work and to how I approach my coding projects.

2 Lecture principles

Below, I discuss two topics from the main lectures and one topic from each of the two guest lectures.

2.1 Quality assurance through testing

In the lectures, we discussed the topic of software testing, and the term *dynamic techniques* was used to refer to all forms of software testing based on actual code execution.

During my time developing research software, I have mostly relied on manual exploratory testing using tools such as a debugger, logging, and real-time graphics. I have probed the software in different ways and compared the results with expected outputs, including outputs from other simulation software.

Automated testing has not been a priority for me. If a component has been shown to work, I expect it to continue working. I find the psychology behind my choices interesting. Why is my standard way of thinking to probe the software while avoiding writing tests? Are test scripts really that important? How would I go about writing relevant tests when the input-output space is impossibly large?

I have motivated this strategy by the small scope and mainly personal use of the software, although I have gradually created tests for some isolated components. These tests are useful when going back and making changes that affect older parts of the code. As the software is shared more widely and used by others, automated testing becomes increasingly important for ensuring that previously working functionality continues to behave as expected.

2.2 The V-model and AI for software engineering

In the lectures, we talked about the V-model as a way to visualize the software development process. The left-hand side moves from user requirements through increasingly detailed specifications and designs to implementation, while the right-hand side moves from component testing toward system testing and user validation [3]. This places coding at the bottom of the "V".

We also considered the V-model in the context of AI driven development and discussed the idea that AI for software development is progressing from the bottom upward. It would follow that AI is best suited for coding tasks and component-level testing, while it is worse at understand user needs. This is not a very surprising conclusion.

However, I think this simplified view underemphasizes how useful AI can be for requirements engineering. It is just that a human is needed in the loop when working with requirements closer to the end user. I found an interesting blog post where the author argues for using the V-model to guide agentic coding and describes what humans and AI does best at each level [5]. The author actually attributes both the bottom side and right hand side of the "V" as AI-work. I

think this depends on the details of the V-model that is considered—an AI is not suitable for user validation if the user is a human.

In my research I have mostly done agentic coding using Codex. I have mainly used it to generate plots, identify bugs, and implement testable algorithms/sub-components. My experience has been good, but I know I should be careful not to over-rely on these systems. I have found that they can fill out the blanks from non-detailed prompts. . However, this may be a bad thing because it hides choices made along the way, including choices about algorithms and APIs. For larger systems, I could see myself using V-model-driven agentic development, provided that I retain a clear understanding of the overall system.

2.3 Requirements engineering

Another topic covered in the first guest lecture was requirements engineering and the cost of defect removal.

I find the idea of requirements engineering interesting. By specifying requirements in advance, it may be possible to prevent errors that would be much more expensive to fix at a later stage [2]. At the same time, it is rarely possible to produce a complete list of requirements covering every scenario. Beyond some point, trying to anticipate every scenario may become less efficient than addressing the problems as they arise. It is a balance that depends on the project.

In my case, I see little reason to spend time on an exhaustive list of requirements for the software alone. However, specifying the known aspects could definitely be useful, and even more useful if put in a bigger context, such as a research paper.

I have found it better to start simple and refactor the code when needed. The downside is that this may require more effort overall. However, I have noticed the opposite extreme in my personal projects, where I try to make a feature as general as possible from the beginning. This is also extra work that may not be needed. If the direction of the project is unclear, I now prefer to start simple, make necessary assumptions, and accept the cost of later rework or defect removal.

2.4 Human aspects in software engineering

In the second guest lecture, one of the topics was how less routine work does not automatically mean better work. When tedious, repetitive tasks are performed by AI, the remaining work tends to be more cognitively demanding. However, there is a limit to how much cognitively demanding work humans can sustain, and repetitive tasks may have an important role in providing variation and recovery. It is therefore necessary to find a balance between variation, cognitive intensity, and recovery.

My subjective experience aligns with this view. I enjoy typing when I know what to type, and programming can sometimes be that way, for example when refactoring code. I often miss this kind of tedious work in my research. Data

collection can provide it, but in my case, it usually involves more planning than repetition. The important thing is still to find variation and recovery in the work.

I do not need AI to automate everything that it could, in principle, automate. I am happy to let it take over the least enjoyable repetitive tasks while keeping some of the routine parts that I enjoy, if only to keep myself motivated.

3 Data scientists, software engineers, and AI engineers

In [4], Kästner separates the two roles *data scientist* and *software engineer* in an AI-enabled software project. The data scientists focuses on machine learning algorithms and model evaluation, which often form the core of the project. The software engineer focuses on the system around the core, including the user interface, scalability, security, user needs, and constraints such as time and budget.

I like this distinction and the picture painted in the book, it captures the specialized way the data scientist thinks while pointing out surrounding aspects that are critical in a large-scale software project. However, this simplified description does not capture the number of specializations required in a real project. Expertise in all the areas surrounding the machine-learning core would generally require multiple people rather than a single software engineer. This is probably deliberate.

The system-wide way of thinking definitely applies to my own research. It is always useful to consider how the research integrates with other components in a full software system, as well as how it affects humans. In my case, this means thinking about control algorithms, interfaces, human impact, and the risk of optimizing one component at the expense of the overall system. For example, a system-identification algorithm cannot be used in real time if it is not fast enough. The surrounding system may also need additional code for detecting sensor failures and responding safely when they occur. The quality of the algorithm alone is therefore not sufficient to determine the quality of the complete system.

I think data scientists and software engineers will increasingly need to learn each other's skill. This applies especially to data scientists, who now have access to powerful agentic coding tools that lower some of the barriers to software engineering. I think knowledge of the "complete" software stack is more useful to data scientists than knowledge of data science is for software engineers. There will still be software without machine-learning components in the future, even though AI for software engineering will significantly change how software is developed. For these systems, a software engineer does not necessarily need expertise in data science.

As more systems are built on top of pre-trained foundation models such as LLMs, it may be useful to define AI engineering as a third role within this pic-

ture, creating a three-layer structure. Such systems include retrieval-augmented generation systems, multi-agent systems in which agents need to communicate, hard-coded prompts, and more. In a software project built with this strategy, the data scientist would work with model selection, adaptation, and fine-tuning rather than training models from scratch. Surrounding this core, the AI engineer would work on systems built on top of the foundation models. MLOps could be the responsibility of either the data scientist or the AI engineer. Surrounding this layer would be the software engineer, with the same responsibilities as described previously.

4 Papers

4.1 Paper 1: Agentic AI Architecture for Evaluating and Improving Reinforcement Learning Pipelines

The core idea of this paper, [6], is the use of a multi-agent system for reviewing how well a code-base complies with RL lifecycle best practices. The authors also look into patching those weaknesses using LLM coding agents.

This is important because many code-bases don't comply with best practices such centralized model versioning, automated evaluation, staged deployments, and retraining triggers. as common RL toolkits offer little guidance on RL lifecycle management. A multi-agent system gives a fast, yet robust way of determining the weaknesses of a systems from which humans and/or AI can take action. Its importance to the engineering of AI systems directly follows from the core idea, to improve reinforcement learning systems over its entire lifespan.

While this project is not directly applicable to my research, the ideas can still be transferred. The more general idea of using AI agents to expose weaknesses in a codebase is definitely applicable and something I already do to some extent. This is one of my favorite ways to use AI in writing as well. A more narrow multi-agent system tailored for system identification purposes is also very possible, identifying other weaknesses than those discussed in the paper. But the most direct applicability comes in the context of a larger system where system identification is just one component. This could in fact be an RL project trained on a virtual world calibrated against real data. More best practices could be identified in this specific context such as model metadata containing calibration details.

To put this idea into the context of a project, lets consider the task of automating log-loading cycles. The AI is trained to identify best grapple pose best on segmented depth image data, grapple the logs safely, and move them to a truck bed balancing speed and energy efficiency. Suppose the AI has already been trained and with some successful tests in the real world. The next step is to start thinking about small scale deployment, but the developers have neglected the RL lifecycle management aspects so far. This is where the ideas from the paper can come in handy. They would use their or someone else's multi-agent

system to find weaknesses in RL lifecycle management. Based on the results, an AI or human can then patch those weaknesses if deemed necessary.

When it comes to adapting my research in the long term to better support the paper's AI-engineering idea, I would start thinking about the full life-cycle of an autonomous system at an earlier stage. I would try to use LLMs/multi-agent systems to critique those aspects in my code.

Evaluating conformity takes significant human effort to find and patch all the weaknesses.

many code-bases have different styles

The paper also looks into how effective LLMs are at patching those weaknesses.

4.2 Paper 2: AutoReSpec: A Framework for Generating Specification using Large Language Models

The core idea of this paper is to improve LLM formal specification generation through a dynamically chosen collaborative pair of LLMs that can recover from failure. The proposed framework is called AutoReSpec [1].

Formal specification generation is important to software engineering becuase writing annotations manually requires expertise and significant effort. If an AI system can generate formal speficications that can then be formally verified, it could increase adoption and ultimately lead to more robust software. This also applies to AI software, although formal verification is more practical for the surrounding pieces, not for the machine learning models themselves

The relation to my research is less direct, as I have not previously worked with formal verification. However, this strategy could be seen as a more formal way to generate tests for an existing code-base, something I have done. It is more interesting to discuss how I could adapt my research to incorporate formal specification generation. I would find it interesting to generate formal verifications for a physics simulator and explore challenges with regards to functions that move beyond simple logic into floating point arithmetic, algebra and calculus. I could tweak my software to use these techniques and something similar to the presented framework.

I think a good fit for formal verification is for safety fallbacks in AI-driven robotics with expensive hardware. Image a project where expensive quadrupeds are used to explore a cave system for a rescue mission. An AI navigation system tells the robot what path to take and an RL controller controls the joints of the robot such that it follows that path. The system also assumes that there might be obstacles in the way that the AI models failed to detect: a too narrow path, or parts that are simply better implemented as override: running low on battery, loss of communication, broken sensors. The formal verification generator would in this context be a complementary layer of safety. It is not responsible for any of the safety fallbacks, it merely generates specifications such that the systems can be guaranteed that the fallbacks behave in a specified way. The formal verification part of such a system is not related to my research, but my research

could fit into the design of an RL-based controller as discussed in previous sections.

5 Evidence, originality, and GenAI-use transparency

Full bibliographic information for all sources is provided in the bibliography. The main sources were the two conference papers selected for discussion [1, 6] and the online textbook chapters on machine learning in production [4]. I also used [3, 2] to back up statements about V-model and defect removal. Finally, the blog post [5] was used as an interesting practical point of view for AI-driven development with the V-model.

I put greatest trust in established peer-reviewed papers and books. Recent conference papers are interesting for discussing newer developments.

I used GenAI for asking questions about unfamiliar concepts (such as formal specification), for brainstorming, source discovery, language checking, and critique. The language checking was applied to text that I had drafted first, and I reviewed the suggested changes before using them. GenAI also helped produce an initial list of potentially relevant papers, but I checked the sources before citing them.

References

[1] Ragib Shahariar Ayon and Shibbir Ahmed. "AutoReSpec: A Framework for Generating Specification Using Large Language Models". In: *Proceedings of the 2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering*. FORGE '26. Rio de Janeiro, Brazil: Association for Computing Machinery, Apr. 2026, pp. 29–39. URL: https://doi.org/10.1145/3793655.3793731.

[2] Barry Boehm and Victor R. Basili. "Software Defect Reduction Top 10 List". In: *IEEE Computer* 34.1 (2001), pp. 135–137.

[3] Kevin Forsberg and Harold Mooz. "The Relationship of System Engineering to the Project Cycle". In: *Proceedings of the First Annual Symposium of the National Council on Systems Engineering*. 1991, pp. 57–65. DOI: 10.1002/j.2334-5837.1991.tb01484.x.

[4] Christian Kästner. *Machine Learning in Production: From Models to Products*. The MIT Press, 2025. ISBN: 978-0-262-38295-3. URL: https://mlip-cmu.github.io/book/.

[5] Ziv Kfir. *Why the V-Model Is the Natural Way to Work with AI Coding Agents*. DEV Community. Published February 16, 2026; accessed August 28, 2026. Feb. 2026. URL: https://dev.to/ziv_kfir_aa0a372cec2e1e4b/why-the-v-model-is-the-natural-way-to-work-with-ai-coding-agents-17g6.

[6] Evangelos Ntentos and Uwe Zdun. "Agentic AI Architecture for Evaluating and Improving Reinforcement Learning Pipelines". In: *EEE/ACM 5th International Conference on AI Engineering - Software Engineering for AI*. Apr. 2026. URL: http://eprints.cs.univie.ac.at/8686/.